



강건우

BACKEND DEVELOPER

CONTACT



Tel. 010-7412-7378



AWARDS

2023 오픈소스 SW 아이디어
해커톤 최우수상

2024 컴퓨터인공지능학부
작품 경진대회 금상

SW 캡스톤 디자인 경진대회
우수상

JBNU SW-AI 공모전 동상

2023 전북지역 대학생
지속가능발전목표
프레젠테이션 경진대회 우수상



CERTIFICATIONS

정보처리기사 2025.09

SQLD(SQL개발자) 2024.09

ADsP 2025.03

리눅스마스터 2급 2024.10

토익스피킹 IM3 2025.09

강건우

백엔드 개발자

동시성·장애·성능 문제를 재현하고, 로그와 지표를 통해
원인을 좁힌 뒤 수치로 검증하는 개발자입니다.

ABOUT ME

- 단순히 동작하는 코드를 만드는 데서 그치지 않고, 문제의 원인을 끝까지 파고드는 개발을 지향합니다.
- 동시성, 외부 시스템 장애, 분산 환경에서 발생하는 문제를 직접 재현하고 Before / After 수치로 검증하는 과정을 중요하게 생각합니다.
- 현재는 Spring 기반 백엔드 경험을 바탕으로 안정적인 서비스와 플랫폼을 설계하는 역량을 넓혀가고 있습니다.

SKILLS

Backend

Java · Spring Boot · JPA · MyBatis

Data / Messaging

PostgreSQL · Redis · Kafka · SSE

Infra / DevOps

AWS · EKS · Docker · Terraform · ArgoCD · GitHub Actions

Test

Testcontainers · k6 · Harness · Contract Test

Monitoring

Prometheus · Grafana

EXPERIENCE

(주)스패셜티

소프트웨어 개발자

2025.11.01 ~ 재직중

EDUCATION

전북대학교

통계학과 · 컴퓨터공학 전공

학점 4.02 / 4.5

2020.03 ~ 2026.02

ACTIVITY

UMC

전국 대학 연합 IT 프로젝트 동아리

2025.03 ~ 2025.08

FIVLO

대학교 창업팀 FIVLO (개발팀장)

2025.07 ~ 2026.02

WHAT I VALUE

REPRODUCE

문제를 재현

MEASURE

지표로 측정

TRACE

원인 추적

PREVENT

재발 방지

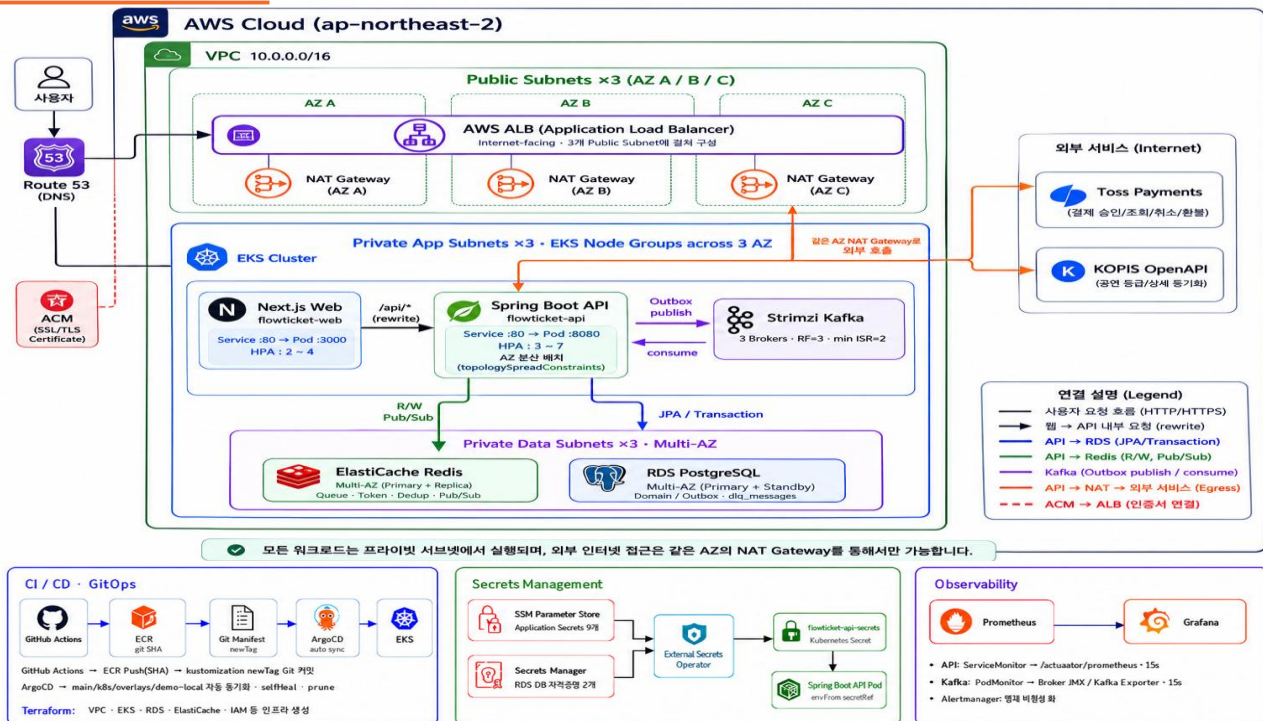


FlowTicket

선착순 예매 시스템 · 개인 PROJECT

2026.06~2026.08

ARCHITECTURE



개요

티켓 오픈 순간 몰리는 요청을 대기열로 제어하고, 좌석 선점부터 주문·결제까지 발생하는 동시성 문제와 중복 요청, 외부 결제 장애를 다루는 선착순 예매 서비스입니다.

Redis 기반 대기열, DB 조건부 상태 전이와 결제 멍등성, Transactional Outbox 기반 Kafka 이벤트 발행, SSE 실시간 알림을 구현했습니다. 또한 계약·정적 가드·통합/E2E 테스트·CI를 연결한 검증 하네스로 변경 사항을 반복 검증하도록 구성했습니다.

SKILLS

Backend

Java 17 · Spring Boot 3.3.5 · JPA · Spring Security · QueryDSL

Frontend

Next.js 14 · TypeScript · TanStack Query · Tailwind CSS

Data / Messaging

PostgreSQL · Redis · Kafka · SSE

Infra / DevOps

AWS · EKS · Docker · Terraform · ArgoCD · GitHub Actions

Test / Harness

Testcontainers · Playwright · k6 · Harness

Monitoring

Prometheus · Grafana

다중 Pod 환경의 대기열 정원 초과 방지

요청 폭주를 Redis ZSET 대시열과 Lua 원자 승격으로 제어

API/DB로 직접 몰리던 요청을 대기열로 흡수하고, 입장 정원을 초과하지 않도록 원자 경계를 다시 설계한 사례

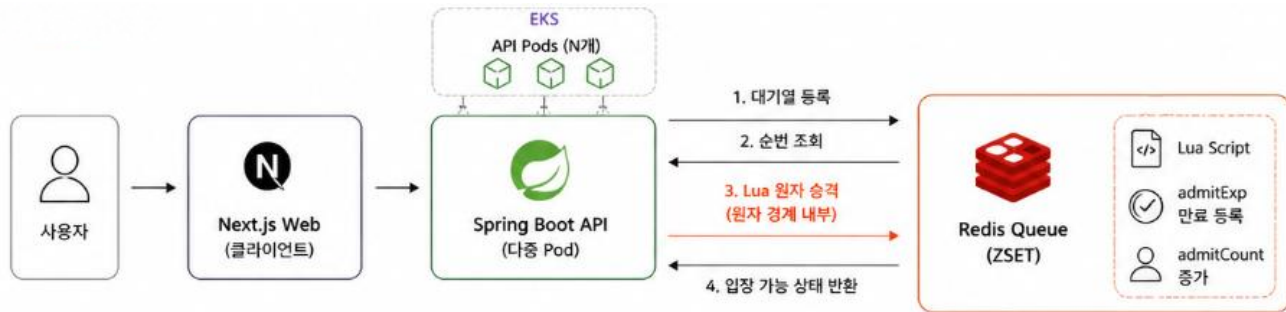
문제상황 / As-Is

1. 티켓 오픈 순간 대량 요청이 API/DB로 직접 유입
2. 여러 Pod가 동시에 승격을 시도하면 정원 초과 입장 가능
3. 초기 test에서 capacity 3, concurrent 10조건에서 over-admit 7건 발생

장애 및 경합 시나리오



To-Be 구조



입장 로직의 원자성을 Redis Lua 내부에서 보장하기 때문에 여러 API Pod가 경쟁해도 안전합니다.

Redis Lua를 선택한 이유

판단 기준	설명
진실원은 Redis 대기열과 정원상태	핵심 대기열 상태를 Redis가 소유하므로 원자성도 Redis에서 보장
ZSET은 순번/정렬/대기열 처리에 적합	진입 순번을 score로 저장해 ZRANK, ZCARD, ZPOPMIN으로 순번 조회와 승격 처리
Lua는 여러 Pod 동시 실행에서도 승격을 원자화	pop + admitCount 증가 + admitExp 만료 등록을 Lua로 원자 수행
단일 worker 직렬화는 확장성 저하	Lua 자체가 다중 Pod 동시 실행에서도 안전하므로 단일 리더로 제한하면 병렬 처리 이점을 잃음
DB 락은 대기열 상태의 소유권이 DB에 있지 않아 부적합	대기열 진실원이 Redis인데 별도 DB 락을 두면 조정 계층과 운영 복잡도가 추가됨
애플리케이션 메모리 락은 다중 Pod에서 불완전	JVM lock은 각 Pod 내부에서만 유효하므로 Pod 간 경쟁을 조정하지 못함

검증 방법

- ✓ Synthetic concurrency test
Testcontainers 기반 동시성 테스트
- ✓ EKS multi-pod spike test
EKS 7-Pod 환경에서 대규모 스파이크 검증
- ✓ 로그 / 메트릭 기반 admitCount 검증
admitCount 로그와 Prometheus
메트릭 샘플링 분석
- ✓ 회귀 테스트로 후속 버그 재현
기존 시나리오를 기반으로 회귀 테스트 수행

결과

① Synthetic concurrency test (개선 결과)

항목	조건 (As-Is)	결과 (To-Be)	비고
capacity	3	3	정원
concurrent	10 threads	10 threads	동시 요청
over-admit	7건 발생	0건	정원 초과 제거

② EKS 7-Pod spike test (운영 환경 검증)

항목	조건	값	비고
API Pods	-	7 Pods	다중 Pod 환경
capacity (정원)	-	100	최종 검증 조건
entrants (입장 요청)	-	3,000명 × 3 runs	스파이크 부하 시뮬레이션
관측 샘플 수	-	총 1,117개	admitCount 샘플 수
관측 최대 admitCount	-	100	샘플링 구간에서의 최대값
결과	-	정원 범위 유지 확인	관측된 모든 샘플에서 ≤ 100

좌석 선점 및 재고 정합성

DB 조건부 UPDATE로 좌석 상태 전이를 원자화

같은 좌석에 대한 동시 요청에서도 초과판매가 발생하지 않도록, 좌석 상태를 DB에 두고 원자적 상태 전이로 경합을 제어

문제상황 / As-Is

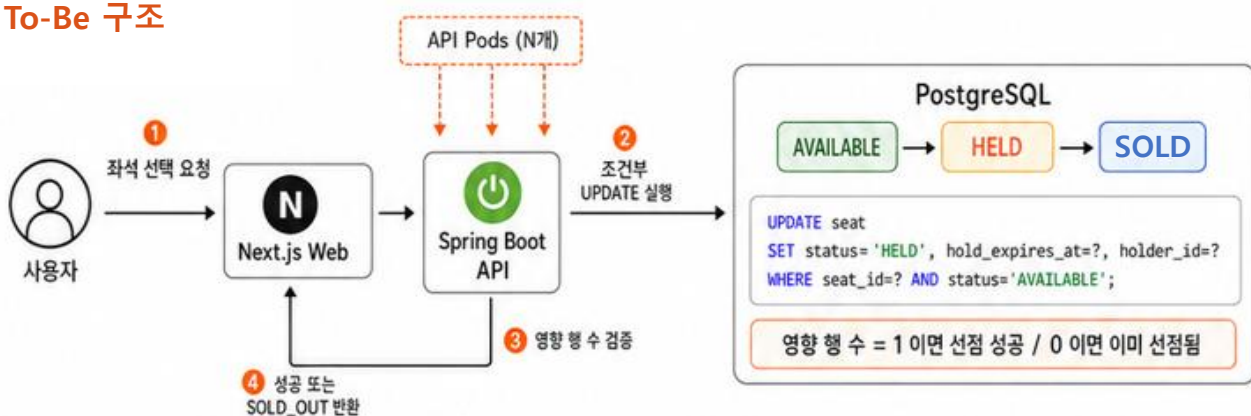
- 여러 사용자가 같은 좌석을 동시에 선택하면 조회 후 저장 방식에서 중복 선점이 발생
- 좌석 재고의 최종기준은 DB 인데 애플리케이션 레벨 판단이 먼저 개입되면 정합성이 흔들릴 수 있음
- 실제 동시성 검증에서 20개 기준 좌석 초과판매 19건 발생
- 목표는 동일 좌석에는 '오직 1명만 성공' 이라는 불변식을 보장하는 것

장애 및 경합 시나리오



조회와 갱신이 분리되어 있으면 race condition이 발생할 수 있다.

To-Be 구조



좌석의 원자성은 DB 조건부 UPDATE가 보장하기 때문에 여러 API Pod 가 동시에 요청을 처리해도 안전합니다.

조건부 UPDATE를 선택한 이유

판단 기준	선택 이유
진실원은 DB 좌석 상태	좌석의 최종 상태를 DB가 소유하므로 원자성과 정합성도 DB에서 보장
좌석별 row 상태 전이에 적합	AVAILABLE → HELD를 조건부 UPDATE 하나로 경쟁 상태와 함께 표현 가능
비관적 락의 대기·경합 비용 회피	요청을 Lock 대기시키기보다 영향 행 수로 성공/실패를 즉시 판별
Redis 분산 락의 이중 상태 관리 회피	Redis lock lease와 DB transaction을 함께 관리해야 하는 복잡도를 추가하지 않음
영향 행 수로 검증 가능	영향 행 수가 요청 좌석 수와 같을 때만 성공시켜 테스트와 회귀 검증이 명확

검증 방법



Synthetic 테스트 (Testcontainers)
1석 × 20스레드 동시 선점 재현하여 초과판매 여부 확인



실 API/k6 테스트
1석 × 100명 경합을 20회 반복하여 응답 코드 분포 및 안정성 확인



DB 사후 검증
응답 코드와 DB 상태를 검증하여 초과판매 여부 확인

결과

항목	AS-IS	TO-BE	비고
1석 × 20스레드 선점 성공 (Synthetic)	20명	1명	동일 조건 재측정 (Testcontainers)
좌석 초과판매 (Synthetic)	19건	0건	동일 조건 재측정 (Testcontainers)
실 API 1석 × 100명 (k6, 20회 반복)	-	1명 성공 / 99명 SOLD_OUT	20회 모두 동일
5xx / 기타 응답 (k6, 20회 반복)	-	0건	정상 경쟁 거절 확인
DB 활성 Hold 중복 (좌석당 Hold ≥ 2)	-	0건	DB 사후 불변식 검사



AS-IS: 조회 → 판단 → 저장 (Check-then-act) 방식으로 초과판매 발생
TO-BE: 조건부 UPDATE + 영향 행 수 검증으로 원자적 상태 전이 보장

주문, 결제, 환불 정합성

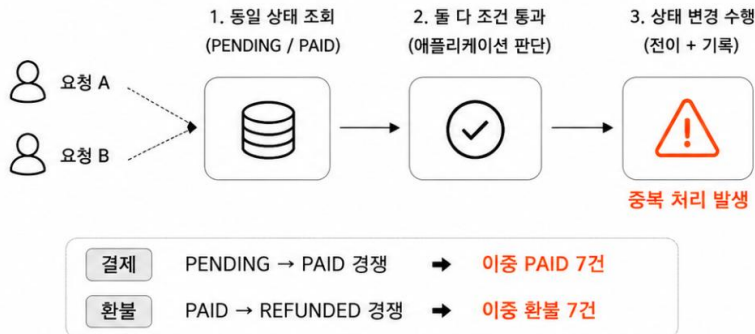
역등 키 + 조건부 상태 전이로 중복 처리 방지

중복 클릭, 네트워크 재시도, 외부 결제 결과 재전송 상황에서도 주문, 결제, 환불 상태가 한번만 안전하게 반영되도록 설계

1. 문제 상황 / As-Is

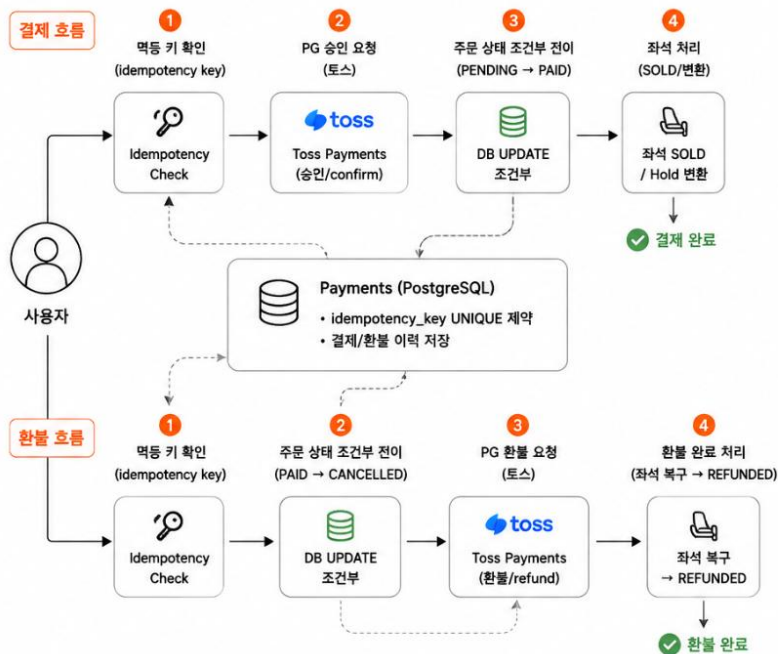
- 동일 주문에 더블클릭, 재시도 등으로
동시 요청이 유입될 수 있음
- 조회 -> 판단 -> 변경'이 분리된 Check-
then-act 구조에서 경쟁 조건 발생
- 동시성 테스트에서 10 스레드 기준
이중 PAID 7건, 이중 환불 7건 발생
- 목표는 같은 결제/환불 요청은 한번만
반영 이라는 불변식을 보장하는 것

2. 장애 · 경합 시나리오



※ 동일 주문에 10 스레드 동시 요청 테스트 결과

To-Be 구조



주문 상태 상태 머신



조건부 전이 / 역등 규칙 (예시)

- 결제 확정: PENDING → PAID
WHERE status = 'PENDING'
- 환불 진입: PAID → CANCELLED
WHERE status = 'PAID'
- 환불 완료: CANCELLED → REFUNDED
- 중복 요청 차단: UNIQUE(idempotency_key)
- 영향 행 수: 1 = 전이 성공 / 0 = 이미 처리됨

※ 동일 주문에 10 스레드 동시 요청 테스트 기준

4. 왜 이 방식을 선택했는가

판단 기준	선택 이유
진실원은 DB 주문 상태	결제·환불의 최종 상태를 DB가 소유
단일 Row 상태 전이	PENDING→PAID, PAID→CANCELLED를 조건부 UPDATE로 원자화
동일 요청 식별 필요	idempotency_key UNIQUE 제약으로 같은 시도의 중복 생성 차단
분산 환경 대응	여러 API Pod에서도 동일 DB 제약과 상태 조건이 일관되게 적용

5. 검증 방법

- 동시성 재현
동일 주문에 10개 요청을 동시에 실행 (결제/환불 각각)
- 상태·데이터 검증
PAID/REFUNDED 전이 횟수와 결제·환불 행 개수 확인
- 회귀 검증
역등 키 UNIQUE + 조건부 UPDATE 적용 후 중복 처리 재발 여부 확인

6. 결과

항목	As-Is (개선 전)	To-Be (개선 후)	비고
PAID 전이	8회	1회	조건부 상태 전이
결제 이중 처리	7건	0건	중복 결제 제거
환불 행 생성	8회	1회	조건부 상태 전이
환불 이중 처리	7건	0건	중복 환불 제거

최종 : 중복결제, 중복환불 모두 0건을 확인

DB 변경과 Kafka 이벤트 발행의 일관성

Transactional Outbox + Relay로 DB Commit 이후 이벤트 유실 구간 제거

주문, 결제 상태 변경과 이벤트 발행은 서로 다른 시스템에 걸쳐 있어, DB는 성공했지만 Kafka 이벤트가 사라질 수 있는 실패구간 제거

문제상황 / As-Is



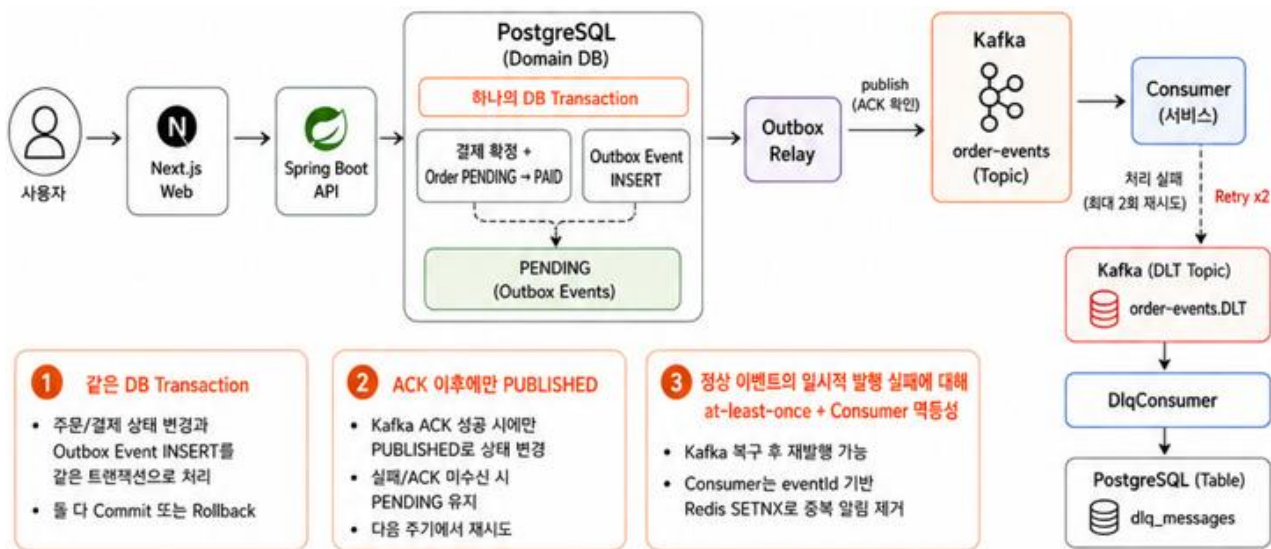
장애 주입 실험 (10개 이벤트)	
DB 변경	10건
Kafka 전달	0건
이벤트 유실	10건

왜 단순 재시도로 해결할 수 없었나

단순 재시도	필요한 것
❌ 실패 이후 복구 근거가 없음 ❌ 어떤 이벤트를 다시 보낼지 알 수 없음 ❌ 재시도해도 유실 이벤트는 복구 불가	✓ 영속적인 복구 근거 보존 ✓ 실패해도 재발행 가능 ✓ 최소 1회 전달 보장

1. 주문, 결제 상태를 DB에 변경한 뒤 Kafka 이벤트를 별도 발행하던 구조
2. DB Commit 이후 Kafka publish에 실패하면 DB에는 반영되었지만 이벤트는 유실
3. 프로세스가 종료되면 무엇을 다시 발행해야 하는지 알 수 없어 재시도만으로는 복구 불가

To-Be 구조 (Transactional OutBox + Relay)



장애별 처리 전략

장애 유형	처리 전략
Kafka 일시 장애 / Timeout	Outbox Event를 PENDING으로 유지하고 다음 주기에서 재시도
Kafka 발행 성공 후 PUBLISHED 기록 전 장애	이벤트가 다시 발행될 수 있음 → 중복은 Consumer의 eventId 역동 처리
역직렬화 불가능한 Poison Outbox Event	DEAD로 격리하여 Relay 전체 정제 방지 (이후 운영자가 Requeue / Discard)
같은 Aggregate의 후속 Event	선행 Event가 DEAD 상태이면 후속 Event는 순서 보장을 위해 보류
Consumer 처리 실패	Consumer Retry 후 Kafka DLT로 이동
운영자 복구	DEAD Event에 대해 Requeue / Discard 가능 제공

검증

항목	AS-IS (기존)	TO-BE (Outbox)
실행 이벤트 수	10건	10건
DB 변경	10건	10건
Kafka 전달	0건	10건
Outbox PENDING	-	10건
이벤트 유실	10건	0건

검증 포인트

- ✓ Kafka 장애 상황에서도 Outbox Event가 DB에 남아 있는가?
- ✓ Kafka 복구 후 모든 이벤트가 실제 Topic으로 전달되는가?
- ✓ Consumer에서 중복 이벤트가 처리되어도 정상 동작하는가?

Kafka 발행 실패 상황에서 이벤트 유실 10건 → 0건

장애 중 10건을 Outbox PENDING으로 보존하고, Kafka 복구 후 전건 재발행

PG 승인 후 DB 롤백 불일치 복구

PENDING/EXPIRED 주문을 주기적으로 조회하여, PG에만 남아있는 승인건을 식별하고 refund로 취소

문제상황 / As-Is

PG 승인 성공 이후, 즉시 보상(finalizePaid) 실행 전에 프로세스가 Crash 되어 DB가 Rollback되면 PG에는 승인만 남아 금전 불일치가 발생합니다.



결과

- PG(Toss) : 승인 상태 존재 (취소되지 않음)
- 내부 DB : Order = PENDING → 이후 EXPIRED 가능
Payment 레코드는 Rollback으로 사라짐
Seat/Hold의 확정 상태 전이는 Rollback됨(미확정)
- 외부와 내부 상태가 불일치 → 금전 불일치 발생

왜 단순 재시도로 해결할 수 없었나



내부 DB만으로 외부 승인 여부 파악 불가
DB에는 승인 흔적이 없으므로,
PG에 실제 승인이 남아 있는지 확인할 수 없음



외부 시스템을 직접 조회해야 함
PG(Toss)에 orderid로 inquire를 수행해야
실제 승인 잔존 여부를 확인하여 정확한 복구 가능



즉시 보상 전 장애 가능
finalizePaid 즉시 보상(1차 방어) 전에 실패하면
PG 승인만 남을 수 있어 Reconciliation(2차 안전망)이 필요

To-Be 구조 (Reconciliation)



선택 이유 및 검증

왜 Reconciliation을 선택했는가

- PG 승인 잔존 여부는 내부 DB만으로 판단 불가하여 반드시 PG를 직접 조회해야 함
- 즉시 보상만으로는 보상 전 장애 케이스를 커버할 수 없어 2차 안전망이 필요
- Order 상태를 보존하면서 외부 승인만 취소하므로 비즈니스 규칙을 안전하게 유지할 수 있음

검증 방법

- 통합 테스트 (PostgreSQL + Mock PG)
→ PG 응답에 따른 Reconciliation 동작 검증
- Reconciliation 메인 로직 검증
→ approved=true 인 경우 refund 호출 확인
- 스케줄 진입 조건 검증
→ grace, 상태(PENDING/EXPIRED) 기준 동작 확인

검증 결과 (통합 테스트 4개 시나리오)

시나리오	검증 내용
1) PENDING + PG 승인	inquire 응답 approved=true → refund 1회 호출 확인
2) EXPIRED + PG 승인 없음	inquire 응답 approved=false → refund 미호출 확인
3) PAID 주문	PG 조회(inquire) 및 refund 미호출 확인
4) grace 내 PENDING 주문	grace 조건으로 PG 조회 미호출 확인

운영 장애 탐지 자동화

프로메테우스 룰 + Alertmanager + Slack으로 대시보드 상시 확인 없이 핵심 장애 신호를 자동 전달
부하테스트 중 API HPA 최대 replica 도달 및 Cluster CPU Overcommit을 Prometheus로 감지해 Slack으로 알림을 전송하고, 부하
해소 후 RESOLVED까지 정상 수신되는 모니터링 파이프라인을 검증

문제상황 / As-Is



대시보드 수동 확인 의존

- 장애가 발생해도 Grafana를 직접 확인해야 인지 가능
- 사람이 보고 있지 않으면 탐지 능력 부족 가능성 있음



5xx로 드러나지 않는 장애 존재

Outbox DEAD, DLQ 적체, Redis fail-open 등은
요청이 200으로 끝나 일반적인 5xx 지표만으로는
탐지 어려움



EKS 기본 알림 노이즈 발생

- KubeSchedulerDown, KubeControllerManagerDown 등
운영에 직접 영향이 적은 기본 알림이 Slack으로 유입되어
핵심 장애 신호가 묻힐 수 있음

To-Be



사용자 요청
증가 / 장애 발생



서비스 / 인프라
이상 징후 발생



규칙 평가 후
조건 충족

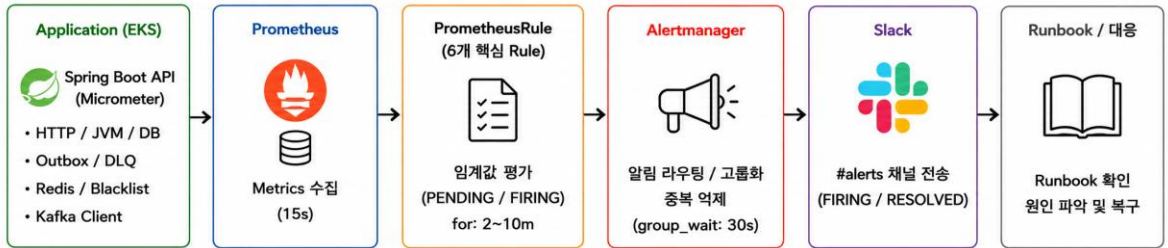


Slack으로 자동 알림
(FIRING / RESOLVED)



대시보드 상시 확인에 의존하던 운영에서,
핵심 서비스 장애 신호를 자동 탐지하고 Slack으로 전달하는 체계 구축

구조 및 알림 흐름



핵심 알림 규칙(총 6개)



※ 임계값은 서비스 특성과 트래픽 규모에 맞춰 조정 가능

왜 6개만 선택했나?

- ✓ **사용자 영향이 큰 신호 우선**
API 실패, 이벤트 폭증, 위크 부하시 직방처럼
사용자 요청 처리에 직접 영향을 주는 신호를 우선 선택
- ✓ **Silent Failure 포함**
일반 Scripting로 드러나지 않는 Outbox, DLQ,
Blacklist 실패를 별도 감지
- ✓ **즉시 행동 가능한 신호만 Alert화**
알림을 받았을 때 운영자가 바로 원인 파악·대응할 수
있는 구체적인 정보
- ✓ **노이즈 최소화**
커스텀 FlowTicket 6개 Rule에서만 p99, HPA max 등
대시보드상 신호를 제외하고, DSC의
KubeSchedulerDown / KubeControllerManagerDown
등 노이즈성 기본 알림은 비활성화하여 핵심 알림만 전달

검증



CI 검증

- promtool check rules
- promtool test rules
- amtool check-config
- amtool config routes test



운영 확인

- 실제 Slack 채널에서
FIRING / RESOLVED 수신 확인
- Prometheus → Alertmanager → Slack
전체 경로 정상 전달 확인



기본 알림 노이즈 정리

K8s Managed Control Plane 특성으로 인해
없는 기본 경고 KubeSchedulerDown,
KubeControllerManagerDown은 비활성화하여
운영 노이즈 감소

Slack 메시지

FlowTicket Alertmanager 백 오전 7:51

[FIRING:1] warning — KubeCPUOvercommit
Cluster has overcommitted CPU resource requests. Cluster has overcommitted CPU resource requests for Pods by 0.70 CPU shares and cannot tolerate node failure.
runbook

7:56 [FIRING:1] warning — KubeHpaMaxedOut
HPA is running at max replicas HPA flowticket/flowticket-api has been running at max replicas for longer than 15 minutes on cluster. runbook

[RESOLVED] warning — KubeHpaMaxedOut
HPA is running at max replicas HPA flowticket/flowticket-api has been running at max replicas for longer than 15 minutes on cluster. runbook

[RESOLVED] warning — KubeCPUOvercommit
Cluster has overcommitted CPU resource requests. Cluster has overcommitted CPU resource requests for Pods by 0.50 CPU shares and cannot tolerate node failure.
runbook

부하테스트 중 API HPA 최대 replica 도달 및 Cluster CPU Overcommit을 Prometheus로 감지해 Slack으로 알림을 전송하고, 부하 해소 후 RESOLVED까지 정상 수신되는 모니터링 파이프라인을 검증

CI 파이프라인 병목 제거

테스트 인프라 재사용과 불필요한 Kafka 초기화 제거로 CI 피드백 시간을 23분대 -> 3분대로 단축
느린 테스트 파이프라인을 계속하고 병목구간을 두 차례에 걸쳐 제거하여, 개발자가 PR 결과를 더 빠르게 확인할 수 있는
실행환경을 구축.

문제상황 / As-Is



1. PR 기준 CI 실행 시간이 23분대로 길어 개발 피드백이 느렸음
2. 테스트 로직보다 테스트 인프라 초기화 비용이 더 큰 병목이었음
3. 병목 위치를 모른 채 최적화하면 개선 효과를 검증하기 어려웠음

To-Be



테스트 본문보다 컨테이너 / Spring Context 초기화와
불필요한 Kafka 연결이 느렸고, 이를 단계적으로 제거했습니다.

개선 흐름

1차 개선

공통 테스트 인프라 재사용



- PostgreSQL / Redis Testcontainers를 매 테스트마다 새로 띄우지 않고 공유 재사용
- Spring Test Context 재사용이 가능한 구조로 정리
- Kafka가 필요 없는 테스트(22개)를 분리하여 컨테이너/Context 재사용 적용
- Kafka 사용하는 테스트(4개)는 독립 유지



결과 (1차 측정)

23.47분 → 9.35분 (약 60% 단축)

2차 개선

비Kafka 테스트의 KafkaAdmin 병목 제거



- 1차 개선 후에도 backend job이 9 ~ 10분대로 critical path → 재개선
- Gradle 로그로 'test' 구간 분석
- test 526s vs JUnit XML testcase 한 58.1s
→ 467.9s 미귀속 구간 재계속
- Spring Context 부팅 14회 전체 합 446.3s
- 그 중 비-Kafka Context 9회는 각 41초대, 합 372.8s
- 원인: 비Kafka 테스트에서도 KafkaAdmin이 브로커에 연결해 토픽 자동 생성을 시도하여 약 40초 대기
- 조치: non-Kafka 테스트에서 spring.kafka.admin.auto-create=false 적용
→ 토픽 자동 생성 시도 제거



결과 (후속 측정, median)

9분 46초 → 3분 08초 (약 68% 단축)



기능 테스트를 줄이지 않고, 테스트 실행 구조를 정리해 CI 피드백 루프를 단축했습니다.

검증

1. GitHub Actions 실행 시간 비교 (개선 전/후)
2. Gradle task 로그로 'test' 구간 계속
3. JUnit XML testcase 시간과 Spring Context boot 시간 분리 계속
4. 개선 후 동일 테스트 수(193개) 유지 확인

수치가 아니라 실행 로그와 반복 측정 결과를
기준으로 병목을 제거했습니다.

의미 있었나?

PR 피드백 대기 시간 감소

개발자가 PR 결과를 더 빠르게 확인 가능

반복 초기화 오버헤드 제거

동일 인프라를 매번 새로 기동하는 비용 제거

측정 기반 최적화

데이터로 병목을 찾고, 효과를 수치로 검증

개발 환경 효율 향상

코드 변경 없이도 개발 생산성 향상에 기여

핵심 성과



1차 측정 결과

23.47분 → 9.35분
(약 60% 단축)



후속 측정 결과 (median)

9분 46초 → 3분 08초
(약 68% 단축)



최초 측정 대비 최종

약 87%
단축



테스트 수 유지

테스트 193개 유지
(43 suite / 193 tests)

트래픽 증가에 따른 Pod, Node 자동 확장

Web HPA + Cluster Autoscaler로 애플리케이션과 인프라가 단계적으로 자동 확장

부하 테스트로 병목 티어를 식별하고, Web HPA를 적용해 병목을 해소 및 Pod가 노드 용량을 초과할때 Cluster Autoscaler가 노드를 확장하도록 구축

문제상황 / As-Is



600 rps 부하 테스트 (AS-IS)					
실제 처리율	실패율	Client p95	Dropped	TargetConnectionError	Target_5XX
551 rps	4.06%	10,006ms	1,726	457건	0건

✓ API는 여유가 있었으나, 앞단 Web에서 연결 실패 대량 발생
 ✓ Hikari pending 0, API server p95 15ms (API 병목 아님)

To-Be



HPA 구조 개선

AS-IS	TO-BE
- Web: 2 Pods 고정 - API: HPA 3 ~ 9 - Web CPU request: 100m	- Web: HPA 2 ~ 4 - API: HPA 3 ~ 7 - Web CPU request: 300m - CPU target: 60% - Scale Up stabilization: 0s - Scale Down stabilization: 300s

- ✓ 병목이 아니었던 API의 확장 여유를 Web에 재배분하여 실제 병목 구간인 Web을 수평 확장하도록 구성

HPA 개선 결과(부하테스트)

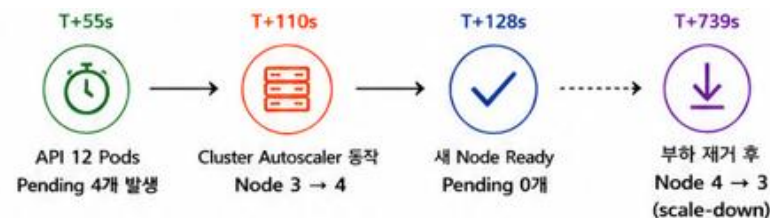
600 rps 구간	AS-IS (Web 2 Pod 고정)	TO-BE (Web HPA 2~4)
실제 처리율	551 rps	599 rps
실패율	4.06%	0.00%
Client p95	10,006ms	123ms
Dropped	1,726	0

TargetConnectionError 457건 → 0건
 (AS-IS 600 rps 구간 457건 / TO-BE 전체 측정 구간 0건)

Cluster Autoscaler 확장 실증

실험 조건

- EKS 1.35 / m6i.large × 3
- Web HPA 2~4, API HPA 3~12 (테스트 중 상황)
- 부하: k6 400 rps × 8분 (클러스터 내부)
- 목적: CA 발동을 위한 Pending 발생 유도



※ Pending Pod 발생 시 Cluster Autoscaler가 새 노드를 추가하여 스케줄링 해소

핵심 성과



Web 수평 확장
 2 Pods 고정
 → 2 ~ 4 Pods
 자동 확장



600 rps 처리 개선
 551 rps
 → 599 rps



사용자 지연 대폭 개선
 p95 10,006ms
 → 123ms



Node 자동 확장 실증
 Pending 4 → 0
 Node 3 → 4 (확장)
 부하 제거 후 4 → 3 (축소)

무중단 배포 안정화

기존 RollingUpdate 전략에서 **presto + terminationGracePeriod** 조정으로 검증조건에서 **5xx 0건** 확인
 배포 중 발생하던 5xx 오류를 줄이기 위해 Web 종료 지연 설정을 조정하고, 실제 Rolling Update 및 Node Drain 상황에서 안정성을 검증

문제상황 / As-Is

장애 및 경험 시나리오



배포 중 5xx 발생

- Web restart에서 13 / 6,001 실패
- 실제 배포 경로에서 16 / 6,001 실패



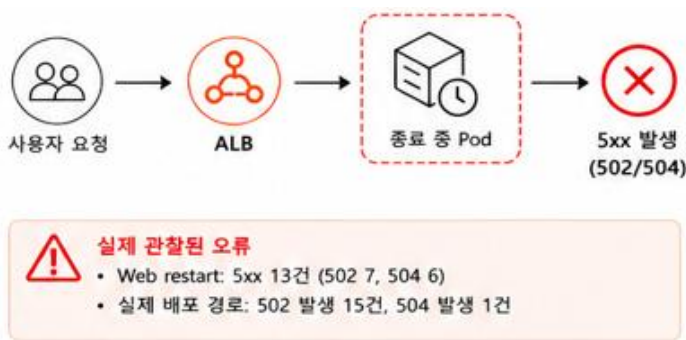
기존 Pod가 빨리 종료됨

- 변경 전 설정
 - terminationGracePeriodSeconds: 30
 - preStop: sleep 5

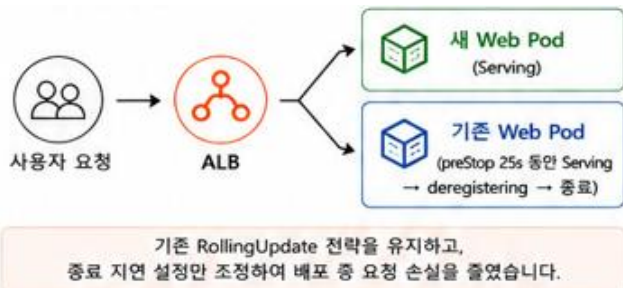


요청 드롭 가능성

- ALB deregistration 전파가 끝나기 전에 Pod 종료 가능



To-Be



인프라 설정

RollingUpdate (기존 유지)	종료 지연 설정 (변경)
- maxUnavailable: 0 - maxSurge: 1	- terminationGracePeriodSeconds 30 → 60 - preStop sleep 5 → 25
기존 RollingUpdate 전략은 유지하고, Web 종료 지연 설정만 조정했습니다.	

왜 이방식을 선택했는지



기존 RollingUpdate 전략 유지

새 Pod를 먼저 준비하고 기존 Pod를 나중에 종료하는 기본 구조 유지



Pod 생존 시간 확대

preStop 25초로 기존 Pod 생존 시간을 늘려 ALB 등록 해제 전파를 기다리도록 함



검증 조건에서 효과 확인

실제 배포와 재시작 경로 모두 Rolling Update 중 5xx 0건 확인

검증 결과

Rolling Update 검증 결과		
구분	변경 전	변경 후
Web restart	13 / 6,001 실패 (502 7, 504 6)	0 / 6,001 실패
실제 배포 경로 (Web + API image)	16 / 6,001 실패 (502 × 15, 504 × 1)	0 / 6,001 실패
개선 후 합계	-	12,002건 중 5xx 0건

※ Web-only image change(개선 전): 64건 실패
 → 수정 후 재검증에서는 Web restart와 실제 배포 경로 모두 Rolling Update 조건에서 5xx 0건 확인

Node Drain 검증 결과 (최신)	
항목	결과
부하 조건	25 rps × 4분
총 요청 수	6,000 requests
non-2xx	0건
p95 / p99	24ms / 40ms
PDB	minAvailable: 2, eviction 차단 확인
Cluster Autoscaler	3 → 4 → 5 scale-out 확인

핵심 성과



배포 중 5xx 제거

실제 배포 경로
16 / 6,001 → 0 / 6,001



Rolling Update 안정성 확보

검증 조건에서
12,002건 중 5xx 0건



노드 축출 상황 대응 검증

Node Drain 상황에서도
6,000건 전부 성공

GitOps 정합성 보장

ArgoCD Self-Heal + HPA ownership 분리 + 배포 순서 가드

HPA가 Pod 수를 소유하도록 Deployment replicas를 제거하고, ArgoCD Self-Heal과 stale SHA 가드로 클러스터 상태와 Git 상태를 일치시킴

문제상황 / As-Is



1. Git에 replicas가 선언되어 있으면
HPA scale-out 후 ArgoCD sync가
다시 덮어써 Pod 수가 흔들릴 수 있음



2. 운영 중 수동 변경이 남으면
Git 상태와 클러스터 상태가
어긋날 수 있음



3. 동시에 여러 커밋이 배포될 때
오래된 SHA가 나중에 manifest를
갱신하면 stale 배포 역전 위험이 있음

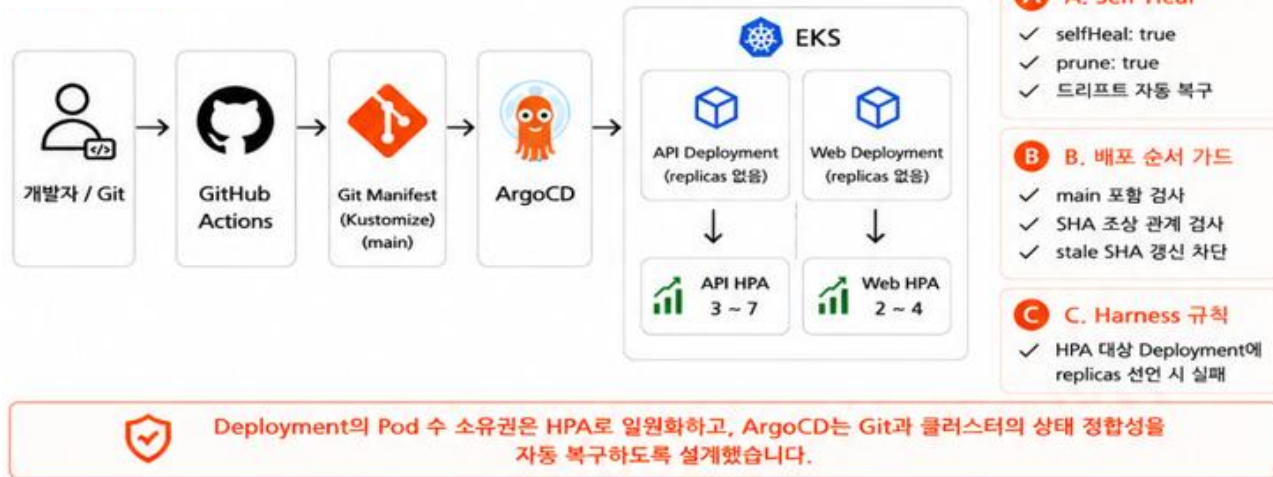
장애 및 정합 시나리오

장애 및 정합 시나리오 (As-Is)



Pod 수 소유권이 HPA와 Git 선언에 동시에 걸리면
의도하지 않은 재조정이 발생할 수 있다.

To-Be 구조



왜 이 방식이 유효했나

- ✓ Pod 수 결정 책임을 HPA로 분리해 autoscaling과 Git 선언 충돌 제거
- ✓ ArgoCD가 설정 드리프트를 자동 복구해 수동 변경 누적 방지
- ✓ stale SHA 가드로 과거 커밋이 최신 manifest를 덮는 역전 방지
- ✓ Git을 단일 진실원으로 유지해 운영 정합성과 추적성 확보

검증 결과

1. ArgoCD Self-Heal 실행
Web replicas 4 → 2 11초 내 복구
2. Config Drift 복구
COOKIE_SECURE 변경 2초 내 원복
3. HPA와 Sync 공존 검증
API 6 → 6 유지 36초 관찰

HPA 대상 Deployment에서 replicas 제거 후,
scale-out 상태(수 6) 유지되어도 Sync 재적용되지 않음을 확인

성과 요약

- HPA ownership 분리
API 3~7, Web 2~4
- 클러스터 드리프트
자동 복구
Self-Heal 검증
- stale 배포 역전 방지
과거 SHA의 manifest
갱신을 가드로 차단